* Last class we were looking at basic considerations in Quantisation. Let us now look at applying Quantisation to LLMs / ML Models.

* Two paradigms
  1. PTQ (Post training Quantisation)
  2. QAT (Quantisation aware training)

PTQ: Take a model and quantise it without training. / ft

TAQ: Mix Quantisation during training.

* When you quantise
  1. Weights only quantisation (W8, W4, ..)
  2. W & A quantisation (W8A8, W8A4, ...)

↓ Weights only: Activations are not quantised.

Workflow:

Quantise Weight → dequantise BF16 → use in down stream

WA:

Quantised W (say INT8) → Activations (next layer) (INT32)

Quant A (say INT8) → Activations (next layer)

Activations BF16

↓ quantise (new grid)

Quantised Activations (say INT8)

Methods of Quantisation

(1) GPTQ (Quantisation Counterpart of SparseGPT)

Recall: Optimal Brain Surgeon

X = calibration data.

W = weight

$\mathcal{L}(X, W)$: loss on the calibration.

$$\min_{\hat{W} \text{ lies on quantisation grid.}} \mathcal{L}(X, \hat{W})$$

① NP-Hard

② Greedy Algo is suitable.

③ full Model is expensive to look at

④ Local loss computation (linear layers)

$\Downarrow$ Lagrange's Multiplier

$$\mathcal{L}(X, \hat{W})$$

$$S_i = \min_{\hat{W}, \hat{W}_i = \text{quant}(W_i)}$$

$\Downarrow$

Saliency of i

$$\delta W = \hat{W} - W$$

$$S_i = \frac{1}{2} \cdot \frac{\delta w_i^2}{(H^{-1})_{ii}}$$

$\Downarrow$

Saliency used to select "greedily" which element can be removed

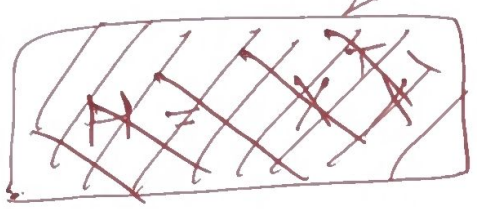
$$\delta w = \frac{\delta w_i}{[H^{-1}]_{ii}} \vec{H^{-1}} e_i$$

$\Downarrow$

Update for rest of the weights.

→ $\mathcal{L}(X, \hat{W}) = \frac{1}{2} \| X\hat{W} - XW \|_2^2$

$n \times d$; $d \times d'$; true weight.



$$\mathcal{L}(X, \hat{W}) = \frac{1}{2} \sum_{j} \frac{1}{2} \| X\hat{w}_j - Xw_j \|_2^2$$

↳ broken down into row-wise terms

$H = X^T X$

(each row) $d \times d$ matrix.

---

1 row

$H^{-1} \times$: $O(d^3) \longrightarrow O(d^3 \times d)$ Naive

$\longrightarrow O(d^3)$ [Exclusion identity]

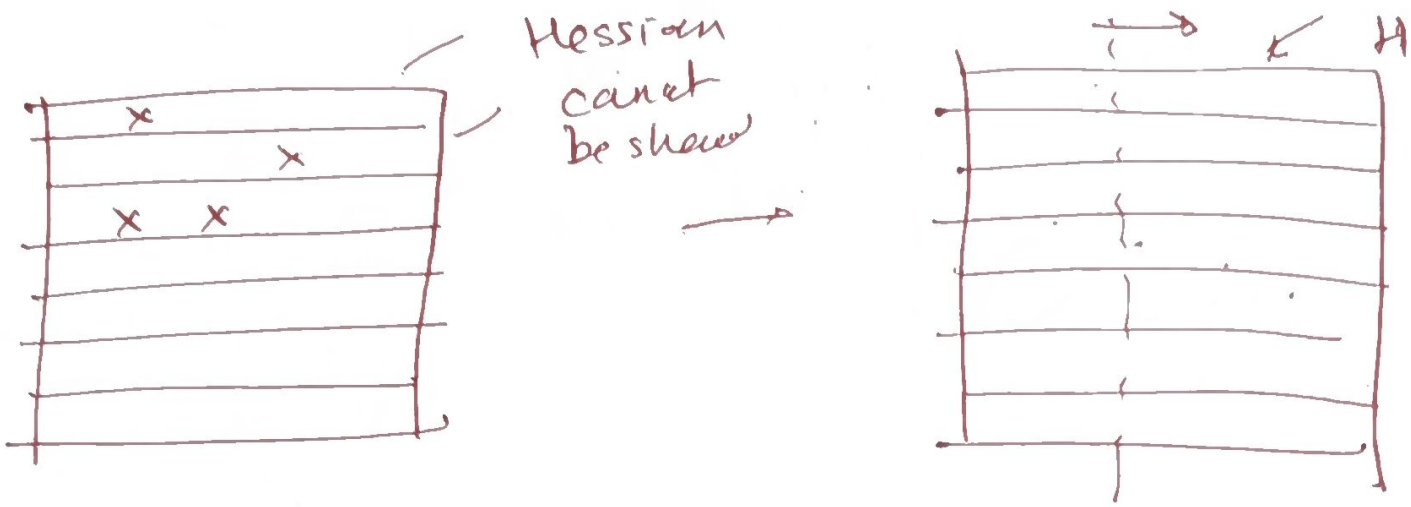
Algo : → ① Select based on scaling
② Remove and update
③ Repeat.

→ $O(d^4)$ for all rows.

Sparse GPT → share H^{-1} across the rows

↓

[Arbitrary order OBS instead of greedy]



$$O(d^4) \rightarrow O(d^3)$$

(Wk1) AWQ : Activation aware Quantisation

GPTQ :→ Compensation algorithm. But the "grid" or quantisation itself is fixed.

AWQ → Adjustments within the granularity

Observation :— "Some Activation Channels have outliers / high magnitudes"

And protecting these weights is more important than others (Sensitivity wise)

A bit "Heuristic" view ← Saliency as a measure of importance

If we use the OBD relaxation.

$$S_i = \delta\omega_i^2 (H_{ii})^2 \rightarrow \|X_i\|_2^2$$

i^{th} dimension

Magnitude of Activation does determine Saliency.

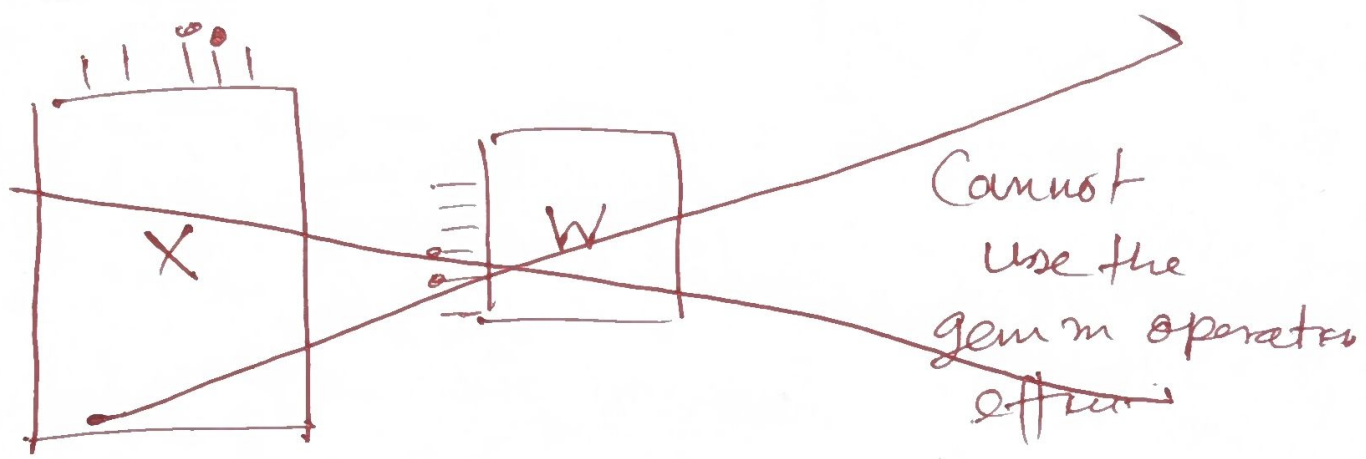
If ~~$\|X_i\|_2^2$~~ i

you identify channels for which magnitude of X_i is large then, ~~can~~ how can we ensure those then weights get ~~better~~ smaller error that

(A) ~~to~~ keep them in ~~do~~ high precision.

↳ Mixed Precision.

↳ Difficult to implement efficiently



~~Channelwise scaling~~

(B) Use the same quantisation precision but reduce errors in quantised for those channels.

Idea :→

$$X[:,i] * \frac{1}{S} \qquad W[i,:] * S$$

look at a single element say of x

$x.w$ / original

x.

w.

$S > 1$

$x \cdot w$ $\quad x \cdot \left[\frac{w}{\Delta}\right] \cdot \Delta$ $\quad \frac{x}{s} \cdot \left[\frac{w \cdot s}{\Delta'}\right] \cdot \Delta'$

Original $\quad$ Quantised

If $s \cdot w$ affects scale.	If $s \cdot w$ does not affect scale.
$x \cdot (w \pm \Delta)$ $\quad \Delta' = s\Delta$	$\Delta' = \Delta$
Same as $x w \pm x \frac{\Delta}{2}$	$x w \pm \frac{x \Delta}{2s}$

$x \cdot w$ $\quad x \cdot \left[w \pm \frac{\Delta}{2}\right]$ $\quad \frac{x}{s} \cdot \left[ws \pm \frac{\Delta'}{2}\right]$

$x w \pm \frac{x \Delta}{2}$ $\quad x w \pm \frac{x \Delta'}{2s}$

General form : $n \times d$ $d \times d'$

X W

$\rightarrow X \cdot \text{diag}(1/s) \cdot \text{diag}(s) \cdot W$

$s^* = \arg\min \mathcal{L}(s)$

$\mathcal{L}(s) = \|$ ~~$Q(X \cdot W \cdot \text{diag}$~~ $\left(X \cdot \text{diag}(s)^{-1}\right)\left(\text{diag}(s) \cdot w\right)$

$\mathcal{L}(s) =$ ~~$Q(X)$~~ $\cdot$

$\| X w - X \cdot \text{diag}(\vec{s})^{-1} \cdot Q(\text{diag}(\vec{s}) \cdot W) \|_2^2$

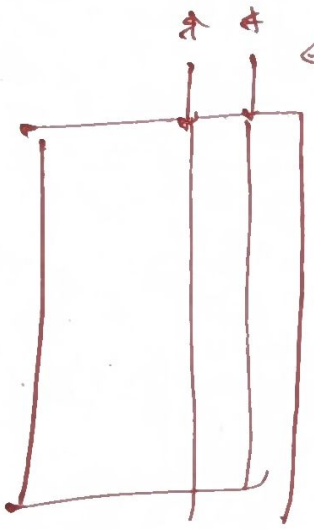
AWQ $\quad \vec{s} = s_X^{\alpha}$ $\quad s_X[i] = \overline{|X[:,i]|}$ (4)

Average chan.

Minimise over (α)

Smooth Quant

W4A4 → Activation has outliers and this actually leads to quantisation being hard.



Ideally :→ channel wise ~~scaling~~ granularity] is inefficient

↳ ~~exe~~

Why inefficient → since this is the inner dimension of Matrix Multiply — cannot Multiply efficient using existing kernels.

↳ ~~Easy to multiply~~

Smooth Quant → transfer hardness of Activation Quantisation to Weight Quantisation

$$\left[\ X\ \right]\ \left[\ W\ \right]$$

$$X\,W \Rightarrow X \cdot \mathrm{diag}(s)^{-1}\,\mathrm{diag}(s)\,W$$

$$\vec{s}_j = \frac{\max(|X_j|)^{\alpha}}{\max(|W_j|)^{1-\alpha}}$$

α = migration strength

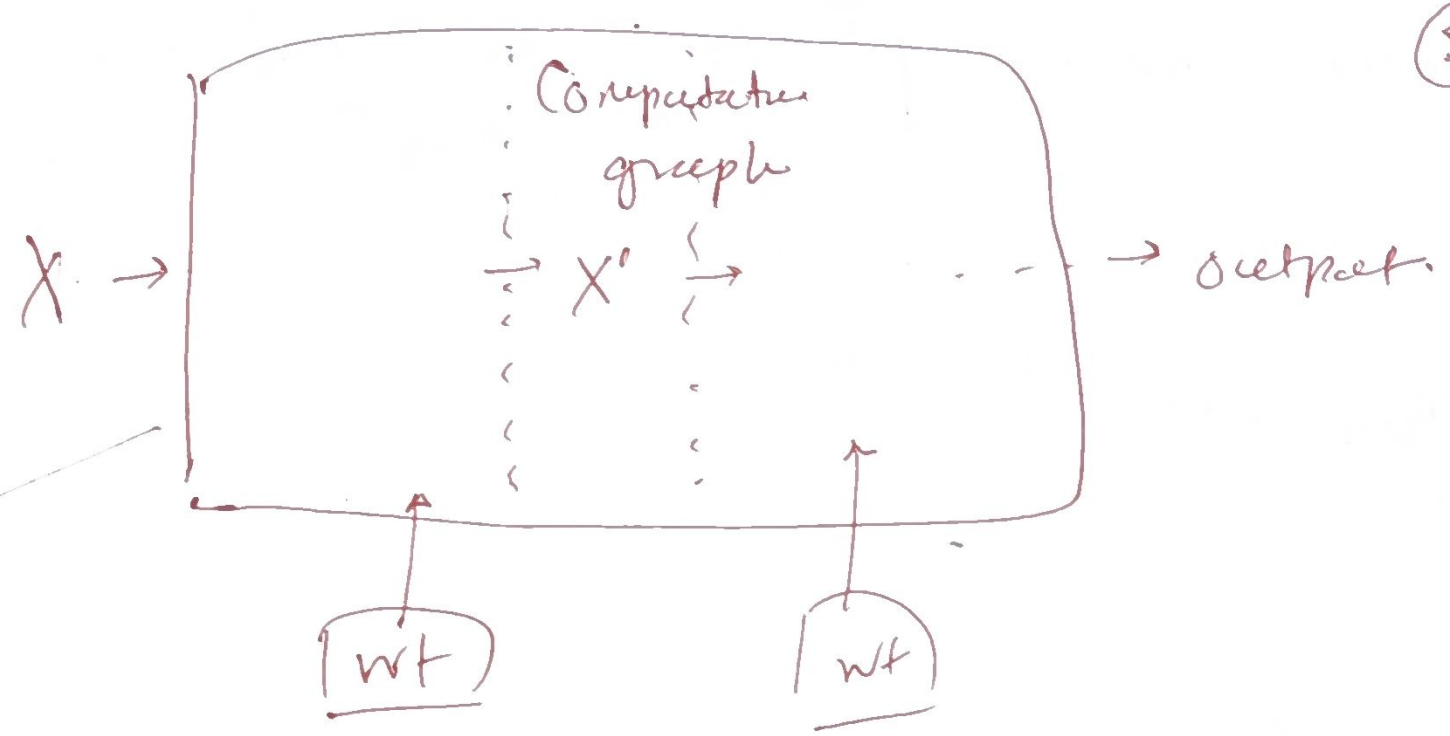
- $\alpha^* = \underset{\alpha}{\arg\min}\ \mathcal{L}(\alpha)$

$$= \underset{\alpha}{\arg\min}\ \left[\ \| XW - Q(X \cdot \mathrm{diag}(s)^{-1})\, Q(\mathrm{diag}(s) \cdot W)\ \|^2\right]$$

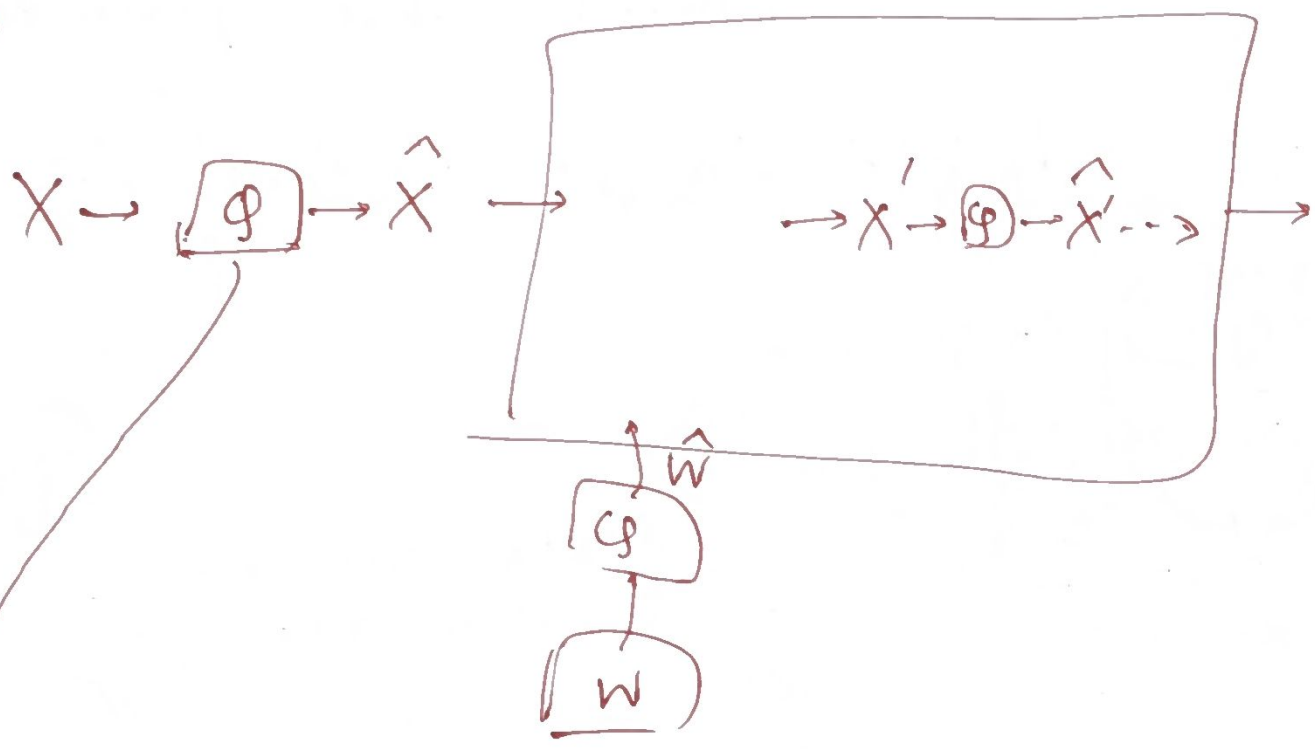
QAT:

PTQ – can affect accuracy adversely

↳ What if we can train the model in quantised aware manner.



Add fake Quantisate Modeals just after weights or activation.



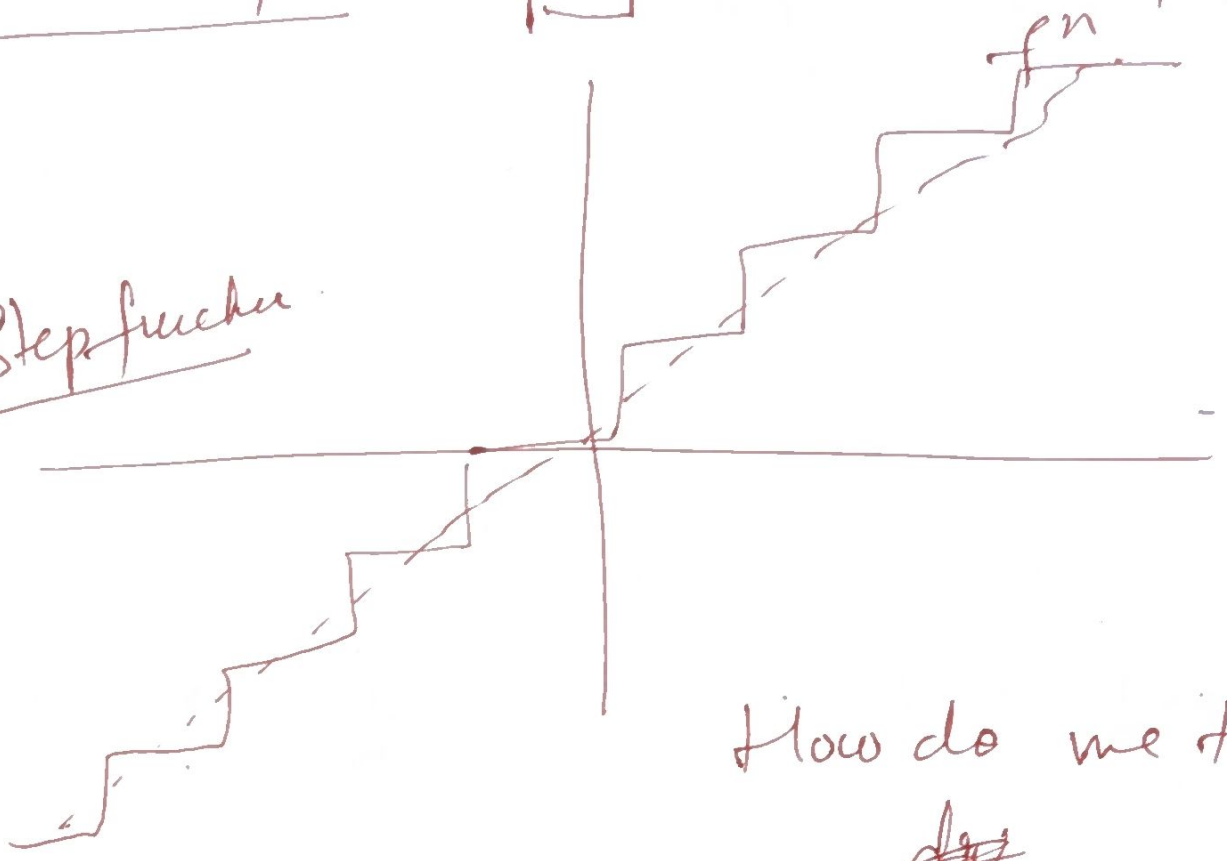
$$\varphi(X) = \varphi^{-1}(\varphi(X)) \quad \text{[therefore]}$$

dequantise. quantise

Forward pass → happens with quantised values.

Backward pass → Uses full precision wts and updates.

Backward pass $\boxed{\Phi}$ is a non-differentiable



Step function

How do we ~~take a~~ ~~der~~ Propagate the gradients?

Answer: Straight through Estimators...

$y = \varphi(x)$

$\left(\frac{\partial y}{\partial x}\right) = 1$ ← Now we can train Normally

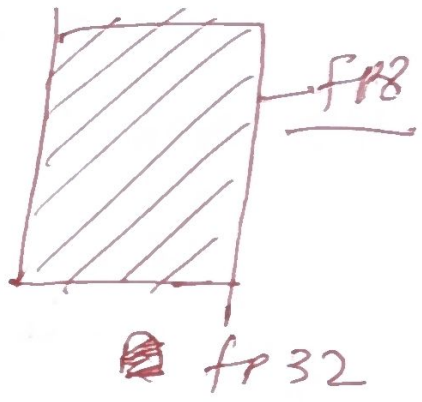
Hardware supported formats

Bf16, FP16, FP32 ← full precision / half precision

① FP8 two formats

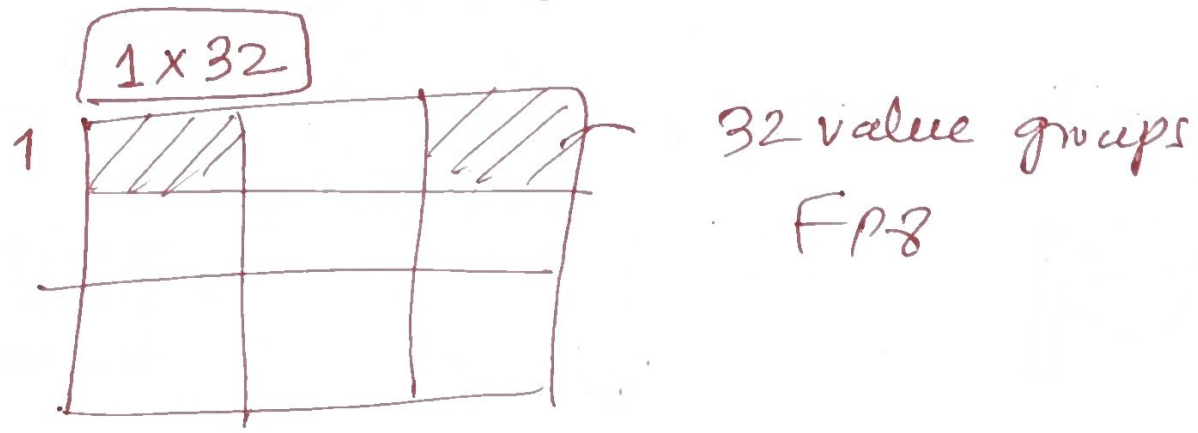
E4M3

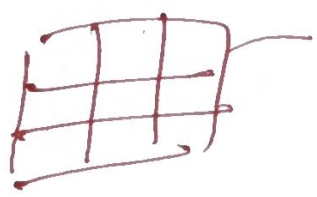
E5M2



Additional Scaler at tensor level FP32

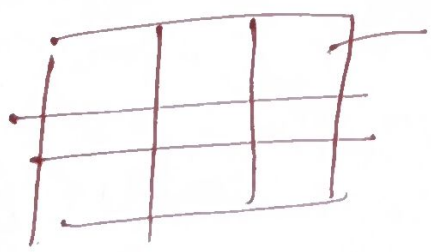
② MXFP8 (Microscaling format – Smaller granularity)

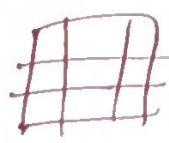




Scales per 32 value groups.

③ NVFP4





~~Using low preci~~

Hardware Supported meaning?

Hardware primitives such as

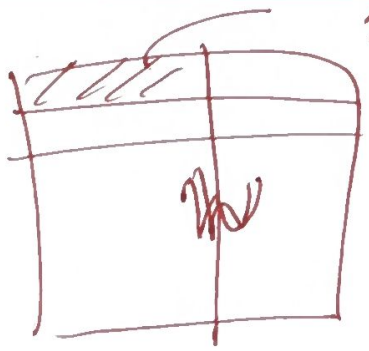
~~funct1(tens1~~

→ MM (tensor 1 , tns 2 , scale 1 , scale 2)

exist Hardware recognis that tensor 1 comes with scale 1

~~No overhead of~~ .

1D tiling + training issues



→ issue ?

↓

$y = Wx$

in backward pass we need W^T !!

So 1D tiles are not usable in backward pass

S8 | 7 : 2D tiles NVFP4 has two versions

2D tiles are used for both training.